扫地僧微软 AI 量化平台 Qlib 教程系列二：滚动训练预测回测与实战

上课点击荔枝微课网址 https://m.lizhiweike.com/liveroom2/27759692
或微信扫描如下二维码上课



目录

第一章 滚动训练与预测原理 4

1.1. 滚动训练与预测原理 4

1.2. Qlib 如何进行每滚的训练集、验证集、测试集时段划分。 5

第二章 安装案例源码 5

2.1. 前提：已安装 qlib，vscode 默认终端设置为命令提示终端 5

2.1.1. 已安装好 qlib，并理解基本的 qlib 用法 5

2.1.2. vscode 设置其默认终端为命令行终端 6

2.2. 案例源码安装 6

2.3. qlib 源码修改 6

第三章 运行源码案例 7

3.1. 设置数据位置，运行案例 7

3.2. yaml 时间设置 7

3.3. 滚动时段生成逻辑 7

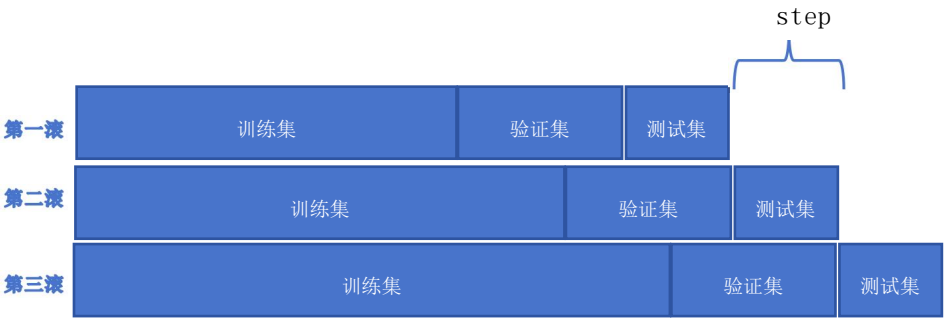
3.4. 训练、预测与回测主流程介绍 7

3.5. 结果分析 7
第四章 关键配置与参数设置介绍 8
4.1. 配置文件 yaml 设置 8
4.2. 滚动步长 step 与预测期 horizon：用于标签定义 label 和截断数据 trunc_days 8
第五章 配置进阶 8
5.1. 线性回归模型不需设验证集 8
5.2. LGBModel 模型需要验证集以便执行早停机制 8
第六章 预测结果放进 backtrader 进行回测 9
6.1. 机器学习滚动训练与预测，得到测试集预测结果 pred.pkl 9
6.2. 安装修改 backtrader 源码、quantstats 源码 9
6.3. Backtrader 执行 TopkDropout 回测主流程 11
6.3.1. backtrader 中获取 pred.pkl 文件 11
6.3.2. backtrader 从 qlib 获取行情数据 11
6.4. TopkDropout 策略原理与实现 11
6.4.1. TopkDropout 策略原理 11
6.4.2. backtrader 中的 TopkDropout 策略实现 12
第七章 单独使用 qlib 内置的 TopkDropout 策略回测 13
7.1. 回测流程 13
7.2. Qlib 中的 TopkDropout 策略实现 13
第八章 幸存者无偏行情数据下载 14
8.1. 下载幸存者无偏行情数据 14
8.2. 下载生成 csi300，csi100，csi500 等成分股股池文件 15
8.3. 后续数据增补流程 15
8.3.1. 增补行情数据 15
8.3.2. 更新成份股股池文件 15
8.4. 用 baostock 新数据测试 Alpha158 因子样本外绩效 15
第九章 实盘操作流程 16
9.1. 更新行情数据（下载数据，更新 qlib 数据），以及股池文件到当日 16
9.2. 执行训练和预测 16

9.3. 执行订单生成程序，生成订单，用于次交易日早上开盘下单 generateOrder.py 16
第十章 滚动训练与预测源码分析 16
10.1. 每滚训练集、验证集、测试集时段定义和各滚训练与预测 16
10.2. 合并各滚预测结果 17
10.3. 生成信号分析与回测结果 17
10.4. handler 对象缓存问题 17
10.5. 简化 yaml 时间设置 17
第十一章 自动生成新因子 17

第一章 滚动训练与预测原理

1.1. 滚动训练与预测原理



pred.pkl 内容样例:记录了每天每只股票的预测收益率评分

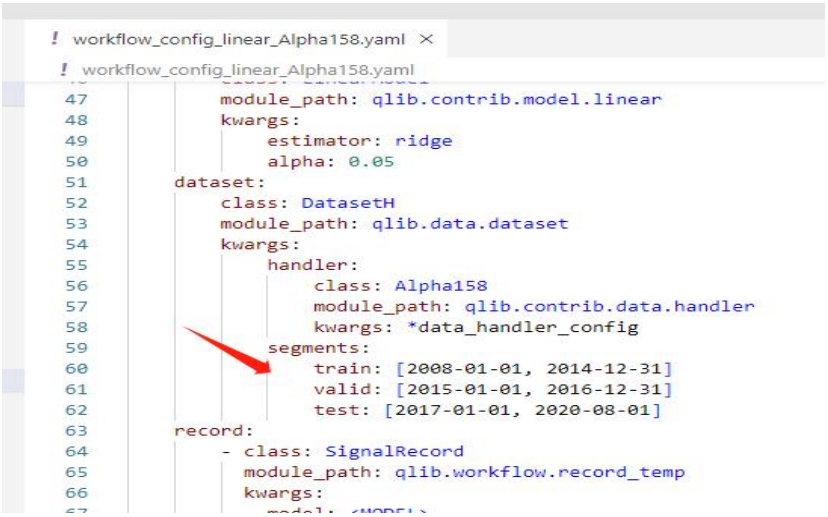
		score
datetime	instrument	
2020-05-20	SH600000	-0.034781
	SH600004	-0.069802
	SH600009	0.113702



合并后的大测试集预测结果 pred.pkl

1.2. Qlib 如何进行每滚的训练集、验证集、测试集时段划分。

在 qlib 的 yaml 配置文件中，用户设置初始整体训练、验证、测试时段划分,如下。用户在程序中设置滚动步长 step，即多少天，



第二章 安装案例源码

2.1. 前提：已安装 qlib，vscode 默认终端设置为命令提示终端

2.1.1. 已安装好 qlib，并理解基本的 qlib 用法

假设用户已经安装好可运行的 qlib，若还没有安装，请参考 www.cctalk.com 上的课程“扫地僧微软 AI 量化平台 Qlib 教程系列一”进行安装（安装视频是免费的）。

2.1.2. vscode 设置其默认终端为命令行终端

本课程使用 vscode 进行开发，设置其默认终端为命令行终端。

2.2. 案例源码安装

2.3. qlib 源码修改

（1）允许输出实验 id

E:\anaconda3\envs\qibrolling\lib\site-packages\qlib\contrib\rolling\base.py

Rolling 类的 init 方法中增加

self._experiment_id = None

再做如下修订

```
201     def _ens_rolling(self):
202         rc = RecorderCollector(
203             experiment=self.rolling_exp,
204             artifacts_key=["pred", "label"],
205             process_list=[RollingEnsemble()],
206             # rec_key_func=lambda rec: (self.COMB_EXP, rec.info["id"]),
207             artifacts_path={"pred": "pred.pkl", "label": "label.pkl"},
208         )
209         res = rc()
210         with R.start(experiment_name=self.exp_name):
211             R.log_params(exp_name=self.rolling_exp)
212             R.save_objects(**{"pred.pkl": res["pred"], "label.pkl": res["label"]})
213             self._rid = R.get_recorder().id
214             self._experiment_id = R.get_recorder().experiment_id # qtb add
215
```

（2）不输出杂乱 git 信息

E:\anaconda3\envs\qibrolling\lib\site-packages\qlib\workflow\recorder.py 文件注释掉 self._log_uncommitted_code()

my_rolling_benchmark.py | base.py | recorder.py × | workflow_config_linear_Alpha158.y

: > anaconda3 > envs > qibrolling > lib > site-packages > qlib > workflow > recorder.py > MLflowRecorder

```
345         # - The logging time may not be accurate
346         self.async_log = AsyncCaller()
347
348         # TODO: currently, this is only supported in MLflowRecorder.
349         # Maybe we can make this feature more general.
350         # self._log_uncommitted_code()
```

第三章 运行源码案例

3.1. 设置数据位置，运行案例

运行案例，查看最终合并预测结果 pred.pkl 文件，查看 qlib 自己的回测结果。

3.2. yaml 时间设置

3.3. 滚动时段生成逻辑

3.4. 训练、预测与回测主流程介绍

3.5. 结果分析

预测结果 pred.pkl 分析

预测信号分析：信息系数、分组收益率等

回测结果分析：topkdropout 策略回测结果分析

第四章 关键配置与参数设置介绍

4.1. 配置文件 yaml 设置

记录器里的 SignalRecord 不需要，可以注释掉

4.2. 滚动步长 step 与预测期 horizon：用于标签定义 label 和截断数据 trunc_days

预测期 horizon 用于标签 label 定义：预测未来几天的收益率

默认 label 定义为 Ref($close, -horizon) / Ref($close, -1) - 1，即 horizon 天后的收盘价相对于明日的收盘价的涨幅。

还用于截断数据 trunc_days

第五章 配置进阶

5.1. 线性回归模型不需设验证集

5.2. LGBModel 模型需要验证集以便执行早停机制

验证集与早停 early stop，迭代训练 n 次若验证集绩效仍然没有改进，则停止训练。

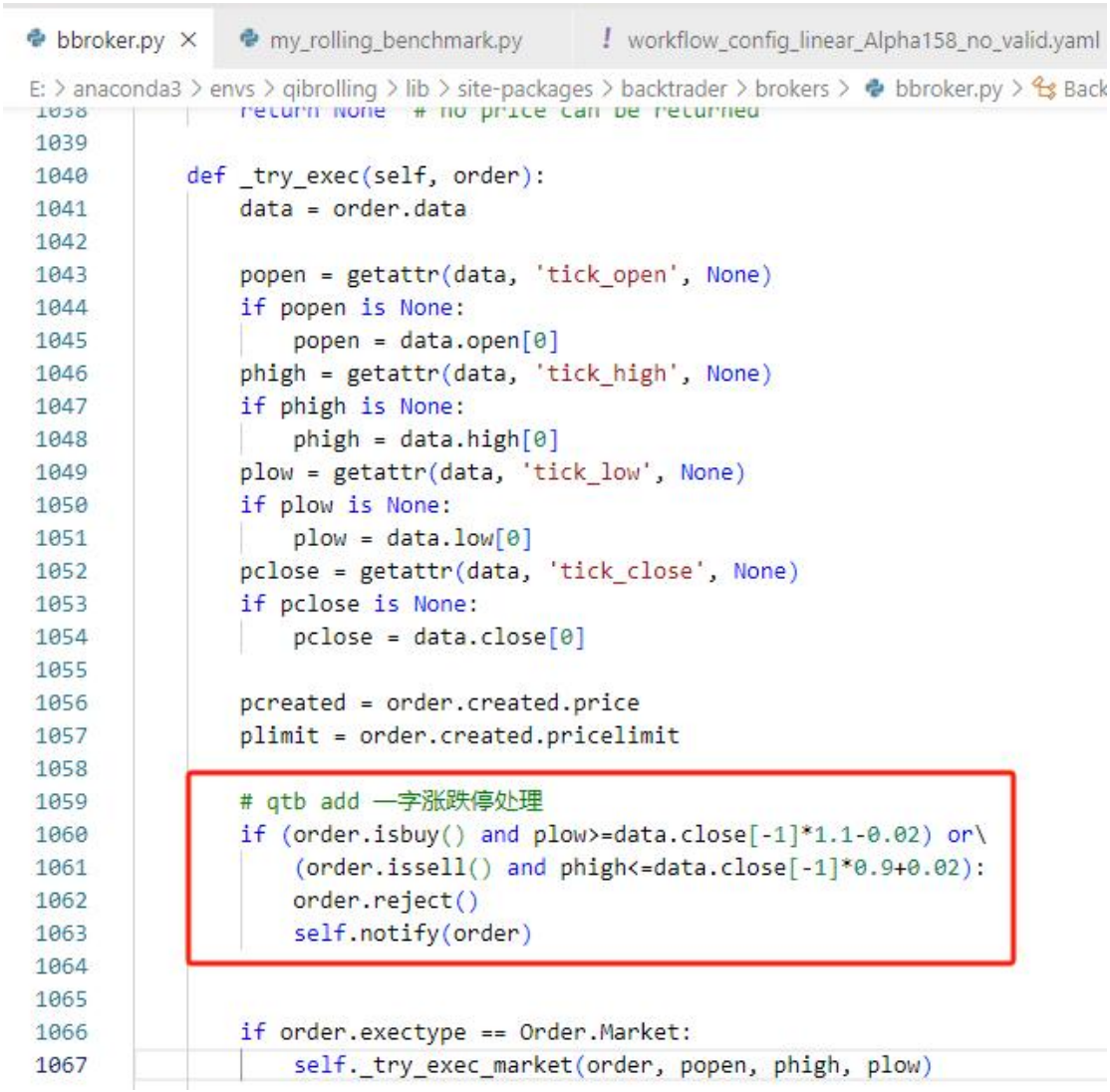
第六章 预测结果放进 backtrader 进行回测

6.1. 机器学习滚动训练与预测，得到测试集预测结果 pred.pkl

yaml 里禁用 qlib 内置回测

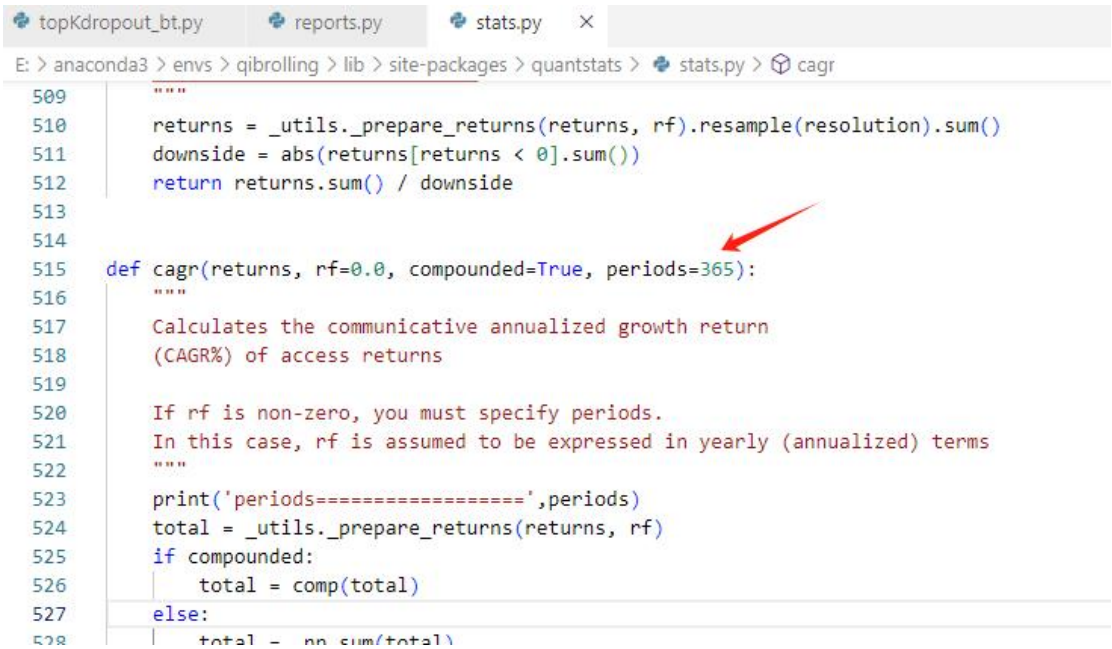
6.2. 安装修改 backtrader 源码、quantstats 源码

（1）安装修改 backtrader 源码处理一字涨跌停



（2）安装 quantstats，修改源码

修改 cagr 函数的默认 periods 为 365，如下



6.3. Backtrader 执行 TopkDropout 回测主流程

6.3.1. backtrader 中获取 pred.pkl 文件

该文件记录了每天每只股票的预测评分（即预测收益率）

6.3.2. backtrader 从 qlib 获取行情数据

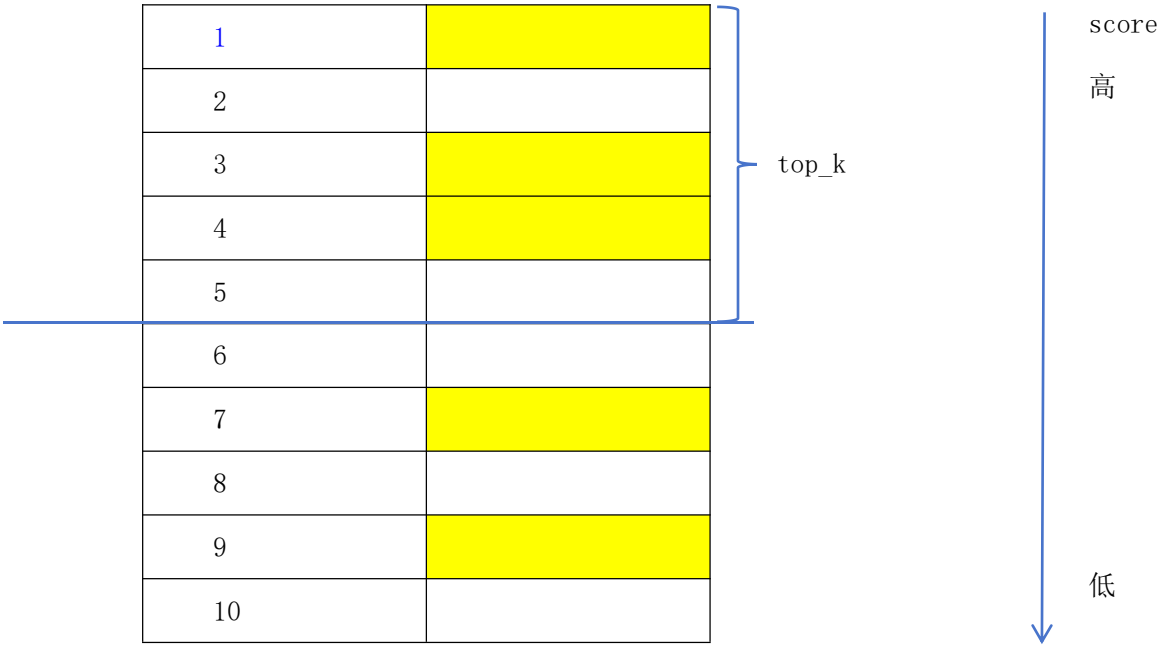
6.4. TopkDropout 策略原理与实现

6.4.1. TopkDropout 策略原理

TopkDropout 策略原理（调仓逻辑与资金分配）：

输入参数举例：top_k=5 保证每日持仓 top_k 支。drop_n=4 每日换仓最多 drop_n 支

以下是包含已持仓股票（黄色）的股池，按预测 score 降序排列。



6.4.2. backtrader 中的 TopkDropout 策略实现

每日调仓：在每日下午收盘后发单调仓（这称为调仓日，也称今日），次日成交。逻辑如下：

pred_df_all 是来自 pred.pkl 的 df

pred_score = self.p.pred_df_all.xs(curr_date),今日有预测值的股池，df 类型，索引为股票代码，列为 score，最好删除即将退市的，以及任何不想买入的股票

以下逻辑可以处理今日持仓中不在 pred_score 股池中的股票，这些股票将排在末尾，最终会进入卖单列表。

```
current_stock_list:今日有持仓的股票代码列表，list 类型
last = pred_score.reindex(current_stock_list).sort_values(by="score", ascending=False,
   na_position='last').index
```

今日已持仓股票列表，index 类型，已按 score 降序排列。若某持仓股票不在 pred_score 中，则该股票排在 index 最后。

```
new = pred_score[~pred_score.index.isin(last)].sort_values(ascending=False).index
```

股池 pred_score 中，去掉已持仓的股票列表，index 类型，按 score 降序

```
today=new[:topk - (len(last) - n_drop)]
```

取 new 的头 topk - (len(last) - n_drop)个股票，index 类型，按 score 降序。这个数量是现有持仓 last 中，去掉最大卖出数量 n_drop，要补齐到 topk，需要买入股票支数。

comb = pred_score.reindex(last.union(today)).sort_values(ascending=False, na_position='last').index
last 和 today 的并集，index 类型，按 score 降序

sell = last[last.isin(comb[-self.p.n_drop:])], last 中位于 comb 中后 n_drop 中的股票，需要卖出。index 类型

buy=today[: topk - (len(last) -len(sell))]: today 中头 topk - (len(last) -len(sell))个股票. 买入数量为现有持仓 last 中，去掉卖出数量 len(sell)，要补齐到 topk，需要买入的股票支数。index 类型

第七章 单独使用 qlib 内置的 TopkDropout 策略回测

7.1. 回测流程

查看每日持仓与订单成交信息

7.2. Qlib 中的 TopkDropout 策略实现

基本同 backtrader 中策略实现一致。

一个差异点：backtrader 中创建订单的时机是调仓日下午收盘后，此时可以拿到当日 ohlc 行情数据，预测数据 pred_score 也是基于当日行情数据的预测。

而 qlib 中创建订单的时机是调仓日上午开盘前，此时不能拿到当日 ohlc 行情数据，预测数据 pred_score 要基于日的行情数据。

第八章 幸存者无偏行情数据下载

8.1. 下载幸存者无偏行情数据

（1）行情从 baostock 下

从 akshare 获取历史退市股代码如下

"stock_info_sh_delist" # 上海证券交易所暂停和终止上市

"stock_info_sz_delist" # 深证证券交易所暂停和终止上市

然后在 baostock 可以下载退市股历史行情。

baostock 每交易日 17:30，完成日 K 线数据入库。

```
import akshare as ak
stock_info_sz_delist_df = ak.stock_info_sz_delist(symbol="终止上市公司")
print(stock_info_sz_delist_df)
```

可以与问财的退市股比较，相差不大

https://www.iwencai.com/unifiedwap/result?tid=stockpick&qs=box_main_ths&w=%E9%80%80%E5%B8%82%E8%82%A1&sign=1700826648217

（2）行情从通达信下

其中退市股代码从 akshare 或 D:\new_tdx\T0002\hq_cache\pttab.dat 获取，然后退市行情看看能不能从通达信下。

8.2. 下载生成 csi300，csi100，csi500 等成分股股池文件

使用脚本 G:\qlibrolling\scripts\data_collector\cn_index\collector.py

源码要修订

cn_index\collector.py 中修订 CSIIndex.get_changes()方法，在其中加入 time.sleep(3)，防止远程限流出错，有时还是出错，可能是当天访问太多，被限流了，换一天再试。

8.3. 后续数据增补流程

8.3.1. 增补行情数据

qlibrolling\scripts\data_collector\baostock\myfetch_baostock_data_append.py

运行 myfetch_baostock_data_append.py 程序，它做如下事情：

（1）本地增补上次更新时已有股票的后续新数据

（2）上次更新数据之后新上市股票的 ipo 以来的数据入 qlib 库

（3）更新日历 day.txt，全市场股池 all.txt

8.3.2. 更新成份股股池文件

若有成份股更新，则执行脚本

G:\qlibrolling\scripts\data_collector\cn_index\collector.py

8.4. 用 baostock 新数据测试 Alpha158 因子样本外绩效

第九章 实盘操作流程

假设执行每周再平衡策略。每周五下午收盘结束后，执行新预测，流程如下：

9.1. 更新行情数据（下载数据，更新 qlib 数据），以及股池文件到当日

9.2. 执行训练和预测

运行 my_update_online_pred.py，执行训练和预测，得到当日的预测文件 pred.pkl，该预测文件只含该日一日的预测。

9.3. 执行订单生成程序，生成订单，用于次交易日早上开盘下单 generateOrder.py

实盘与回测的不同点及对策

第十章 滚动训练与预测源码分析

10.1. 每滚训练集、验证集、测试集时段定义和各滚训练与预测

Qlib 自动在每滚的验证集和测试集衔接处间隔 trunc_days=horizon+1 天，防止验证集中收益率预测期延伸到测试集中，出现前视偏差。

滚动任务生成时截断参数设置：trunc_days 默认 trunc_days=self.horizon+1，在 E:\anaconda3\envs\qlib230510\Lib\site-packages\qlib\contrib\rolling\base.py 的 Rolling 类的 get_task_list 方法里设置

task_l = task_generator(task, RollingGen(step=self.step, trunc_days=self.horizon + 1)，写死在这里，不能且无需改变

10.2. 合并各滚预测结果

10.3. 生成信号分析与回测结果

10.4. handler 对象缓存问题

10.5. 简化 yaml 时间设置

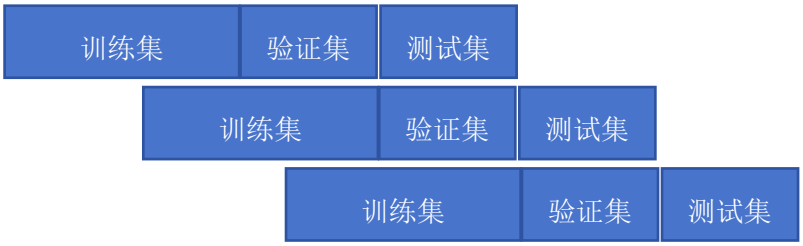
只需设置 segments 部分

```
def _raw_conf(self) -> dict:
    with self.conf_path.open("r") as f:
        config_dict = yaml.safe_load(f)
        config_dict['data_handler_config']['start_time'] = \
            config_dict['task']['dataset']['kwargs']['segments']['train'][0]
        config_dict['data_handler_config']['end_time'] = \
            config_dict['task']['dataset']['kwargs']['segments']['test'][1]
        config_dict['data_handler_config']['fit_start_time'] = \
            config_dict['task']['dataset']['kwargs']['segments']['train'][0]
        config_dict['data_handler_config']['fit_end_time'] = \
            config_dict['task']['dataset']['kwargs']['segments']['train'][1]

        config_dict['port_analysis_config']['backtest']['start_time'] = \
            config_dict['task']['dataset']['kwargs']['segments']['test'][0]
        config_dict['port_analysis_config']['backtest']['end_time'] = \
            config_dict['task']['dataset']['kwargs']['segments']['test'][1]
        return config_dict
    # with self.conf_path.open("r") as f:
    #     return yaml.safe_load(f)
```

第十一章 自动生成新因子

未来开发课程，讲解采用遗传算法，粒子群算法，强化学习等方法自动生成因子的方法。闭环机器学习多因子投资流程。


训练集
验证集
测试集
训练集
验证集
测试集
训练集
验证集
测试集